

Introduction to AWS SageMaker

MLOPs Principles

The following are the principles of MLOps:

- Code, artifact, and experiment tracking
- Cross-team collaboration
- Reproducible results
- Development/Production symmetry
- Continuous integration
- Continuous deployment
- Continuous training
- Module health check

These are major principles of MLOps that remain constant in all the problems we will solve, even if we change the tools or the use case. Hence, it is very important to remember the principles.

MLOps Maturity Levels

The different versions of MLOps are as follows:

- MLOps V1.0: Manually build, train, tune and deploy models
- MLOps V2.0: Manually build and orchestrate model pipelines
- MLOps V3.0: Automatically run pipelines when new data arrives or code changes (deterministic triggers such as GitOps)
- MLOps V4.0: Automatically run pipelines when models start to decay (statistical triggers, such as drift, bias and explainability)

Different companies use different versions of MLOps as per their requirements. It basically depends on the use case or the problem statement as to which version of MLOps should be used.

In each version of MLOps framework, an extra automation level is added.

Startups normally are at version 1.0, and the companies such as Google and Amazon are achieving version 4.0 of the MLOps framework.

Ways to Implement MLOps

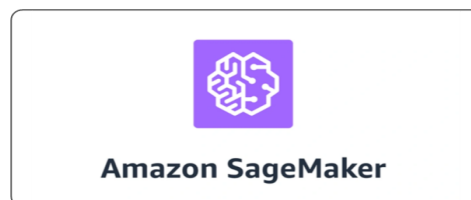
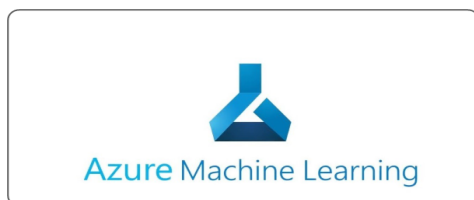
There are two ways to implement MLOps:

1. Open source
2. Managed services

Some of the open source tools available in the market are shown below.



The major and most widely used cloud managed services that can be used for MLOps practices are shown below.



Vertex AI

Azure Machine Learning, Amazon SageMaker and Vertex AI are provided by Microsoft, Amazon and Google, respectively.

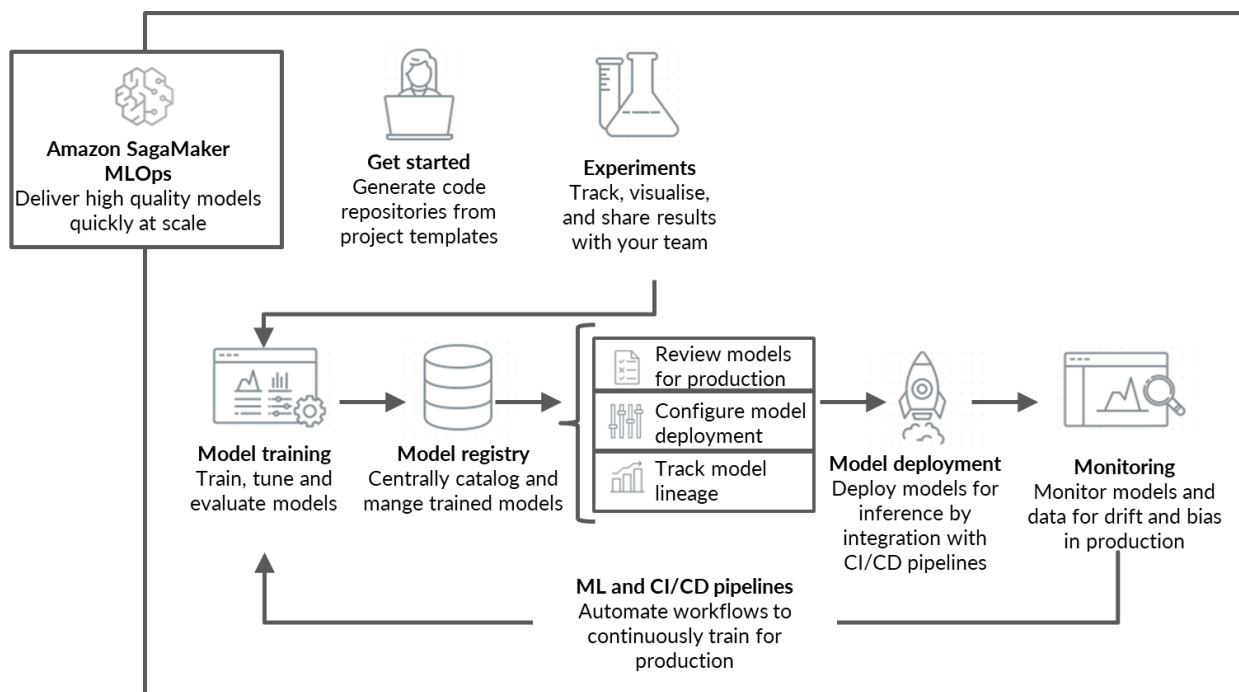
What is SageMaker?

- SageMaker is a cloud managed service provided by Amazon and is used by a myriad of tech giants for its MLOps practices
- Amazon SageMaker is a fully managed machine learning service.
- With SageMaker, data scientists and developers can quickly and easily build and train machine learning models, and then directly deploy them into a production-ready hosted environment.
- It provides an integrated Jupyter authoring notebook instance for easy access to your data sources for exploration and analysis, so you do not have to manage servers.
- It also provides common machine learning algorithms that are optimised to run efficiently against extremely large data in a distributed environment. With native support for bring-your-own-algorithms and frameworks, SageMaker offers flexible distributed training options that adjust to your specific workflows. Deploy a model into a secure and scalable environment by launching it with a few clicks from SageMaker Studio or the SageMaker console.
- Training and hosting are billed by minutes of usage, with no minimum fees and no upfront commitments.

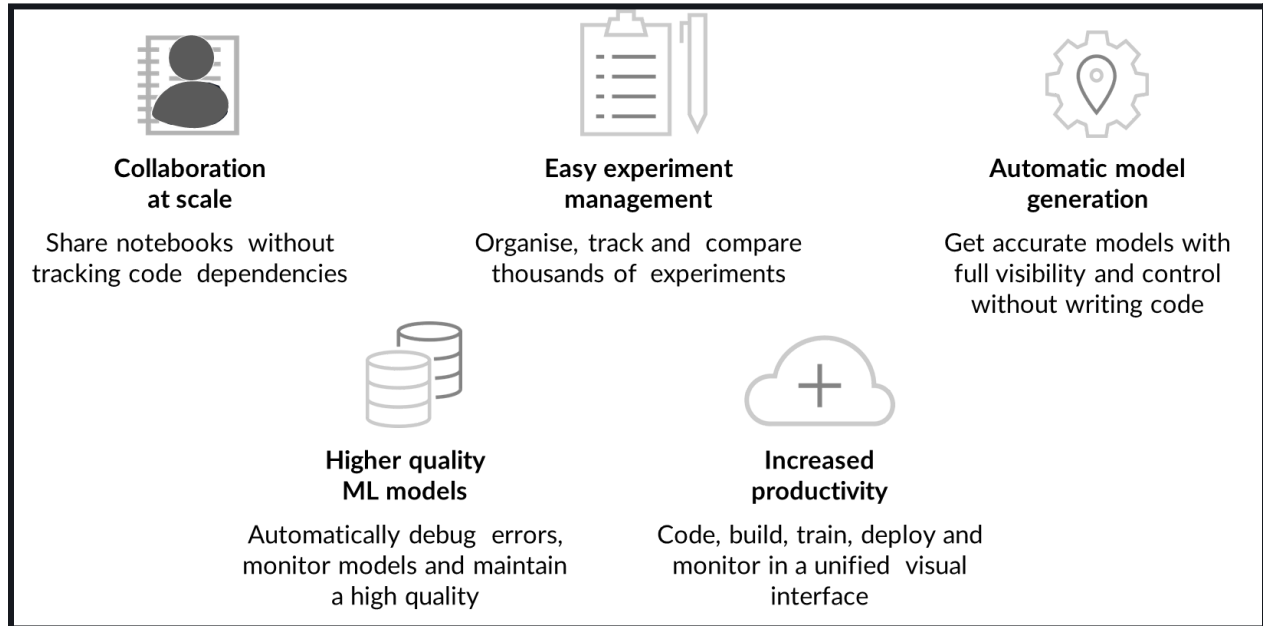
The following are the key features of AWS SageMaker:

- Amazon SageMaker Studio - First fully integrated development environment (IDE) for machine learning. This is similar to Jarvis
- Amazon SageMaker Notebooks - Enhanced notebook experience with quick-start and easy collaboration
- Amazon SageMaker Experiments - Experiment management system to organise, track and compare thousands of experiments. This is similar to MLflow
- Amazon SageMaker Debugger - Automatic debugging analysis and alerting
- Amazon SageMaker Monitor - Model monitoring to detect deviation in quality and take corrective actions. This is similar to Evidently
- Amazon SageMaker Autopilot - Automatic generation of machine learning models with full visibility and control. This is similar to PyCaret

Following is the step-by-step process of model deployment through SageMaker



The features of Amazon SageMaker Studio, which is an IDE for machine learning are as follows:



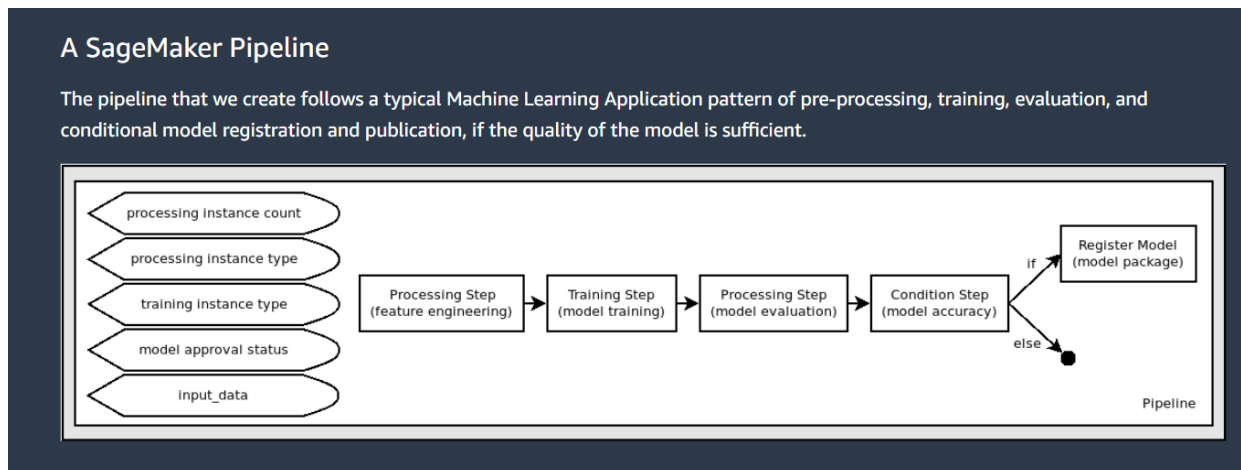
To create an AWS account, you can follow the steps mentioned in this [link](#)

SageMaker tabs and functionalities

- Once you can see your user, you can click on the user and then on the Launch Studio button and you will be redirected to the Amazon SageMaker Studio.
- 'File Browser' option - you can see the folders of your created projects.
- 'Running Terminals and Kernels' - you can use to view the running instances, apps, terminal and kernel sessions and also shut them down.
- 'Git' option - enables you to manage and link your git repositories with SageMaker studio.
- 'SageMaker JumpStart' option - gives you various prebuilt solutions to some of the generic industry problems. You can use these solutions to jumpstart your project and then customise it further as per your requirements.
- 'SageMaker resources' option - provides you with a dropdown menu that has projects, pipeline, endpoints, experiments and trials, etc.
- Projects -> Create Project - you will be able to see different templates to choose from. For our project, we have use MLOps template for model building, training and deployment.
- Once your project is created, you can clone the model build and model deploy repositories to obtain the folder structure.

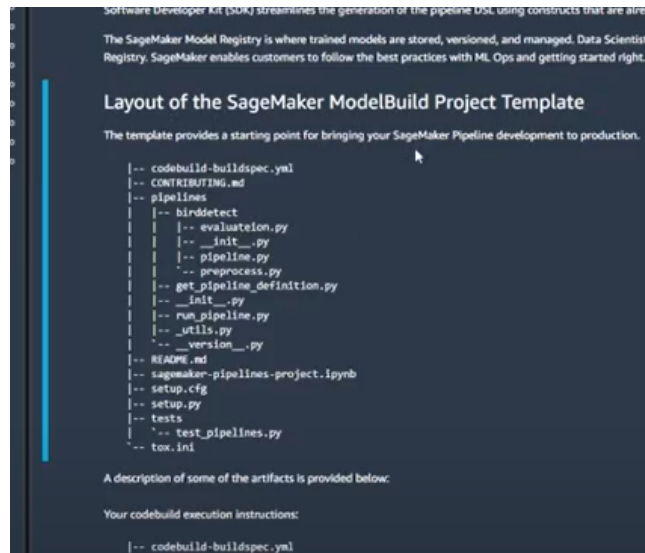
- SageMaker resources -> 'Experiments and trials' option - You can see that for every pipeline execution, a trial with a unique identifier is created.
- SageMaker resources -> 'Model registry' option - allows you to approve one model out of many trained models that will go into production. You can select the 'Model registry' option from the dropdown and select your project. Then, you would be able to see one or more model versions on the main page. Here, you can choose the version and manually change the status of that version from pending to approved or rejected as per your requirement.
- SageMaker resources -> Endpoints option - you can see the various endpoints created in the endpoints option from the dropdown

SageMaker Pipeline



Sagemaker pipeline is a directed acyclic graph of steps and conditions to orchestrate SageMaker jobs and resource creation.

SageMaker modelbuild project template



MLOps Practices and Benefits

Listed below are the MLOps practices and their benefits:

- Code, artifact and experiment tracking
 - **Challenge:** Bridging gap between model building and model deployment tasks
 - **Practice:** Lineage tracking and configuration management
 - **Benefit:** Repeatable process
 - **Solution:** Amazon SageMaker Experiments and Trials
- Continuous integration and deployment
 - **Challenge:** Providing end-to-end traceability
 - **Practice:** Auditable ML pipeline
 - **Benefit:** Improve time to market
 - **Solution:** Amazon SageMaker projects and pipelines
- Continuous training and model monitoring
 - **Challenge:** Continuous delivery and monitoring
 - **Practice:** Maintain model performance over time
 - **Benefit:** Improve time to market
 - **Solution:** Amazon SageMaker model monitor

Open Source Versus Managed Services

The difference between setting up an MLOps system using cloud-based tools and using open source tools are summarised in the table below.

Services	Native cloud-based approach	Open source tools integration
End2End MLOps	Integrated	Plug and play
Time to set up	Less	High
Maintenance of infrastructure	Low	High
Ease of deployment	Low	High
Learning curve	Low	High
IDE Studio support	In-built	Need to be configured
Endpoint deployment	Integrated via SDK	Need to be configured
Pre configured MLOps template	Available	Not available
Companies leveraging	Cloud first companies, which have majority of infrastructure on Cloud	Companies that have on-premises infrastructure

The factors that you should consider when taking the decision regarding build versus buy are as follows:

- **The current stage of your company:** In the beginning, you might want to leverage vendor solutions to get started as quickly as possible so that you can focus your limited resources on the core offerings of your product. As your use cases grow, however, vendor costs might become exorbitant and it might be cheaper for you to invest in your own solution.
- **Competitive advantages of your company:** If machine learning (ML) infrastructure is something that the company wants to be really good at, then generally, it decides to build it in-house. For companies, where ML infrastructure is not their main focus, they tend to buy them.
- **Maturity of the available tools:** Companies that are early adopters build out their own infrastructure because there are no solutions mature enough for their needs. A few years later, solution offerings mature and new companies can opt for these solutions instead of building everything from scratch.

Productionising a Deep Learning Model

- `default_bucket = sagemaker.session.Session().default_bucket()`
Here, we have assigned a `default_bucket`, which will be the default bucket for this project.
- `base_job_prefix = 'End2End-Bird-detection'`
Here, we have chosen a prefix that will be added before any job we will run during deployment. You can choose any name as per your preference.
- `!wget 'https://s3.amazonaws.com/fast-ai-imageclas/CUB_200_2011.tgz'`
Here, we have downloaded the data from the public S3 bucket.
- `!pip install opencv-python`
We have also installed the opencv python library, which, in addition to other necessary libraries, will be used to process the images.

- `input_manifest_key = f'{base_job_prefix}/unlabeled/manifest/{input_manifest_name}'`
`s3.upload_file(input_manifest_name, default_bucket, input_manifest_key)`
.....
`output_manifest_key = f'{base_job_prefix}/pipeline/manifest/{output_manifest_name}'`
`s3.upload_file(output_manifest_name, default_bucket, output_manifest_key)`

Once the input and output manifest files are created, we upload them to the S3 bucket.

-
`class_file_name = "class_labels.json"`
`with open(class_file_name, "w") as f:`
`json.dump(json_body, f)`

`classes_key = f'{base_job_prefix}/unlabeled/classes/{class_file_name}'`
`s3.upload_file(class_file_name, default_bucket, classes_key)`
.....

We created a `class_labels.json` file and uploaded it to the S3 bucket.

- *def make_template(test_template=False, save_fname="instructions.template"):*
 -
 -

Please note that here we are making an instruction template for human annotation purposes. This step is specific to only this case study, as we have downloaded the data from the public S3 bucket.

- *# image location*
s3_input_data = f"s3://{default_bucket}/{base_job_prefix}/unlabeled/images"
labelled manifest location
s3_input_manifest = f"s3://{default_bucket}/{base_job_prefix}/pipeline/manifest"

We provided the machine with the paths for input data and input manifest files.

- *def serialize_example(image, label):*

feature = {
'image': tf.train.Feature(bytes_list=tf.train.BytesList(value=[image])),
'label': tf.train.Feature(int64_list=tf.train.Int64List(value=[label]))
}

example_proto = tf.train.Example(features=tf.train.Features(feature=feature))
return example_proto.SerializeToString()

Then, we created a serialised string that will give us the image and label of the image.

- *def split_dataset(labels):*
channels = {
"train": [],
"valid": [],
"test": []
}
.....

Then, we split the data into three categories: 'test', 'train' and 'validate'.

- *def building_tfrecord(channels, images_dir):*
 -

In this function, we are building a tf record, which will serialise the images and their labels so we can provide this serialised data to our model. We can also tell how many images were processed by the model.

- At the end of the preprocess.py file, we have the generic name class, which will provide the output file of the data once it is preprocessed.
- Please note the `!pygmentize './pipelines/birddetect/preprocess.py'` will load the preprocess.py file in the notebook for us to view it on the same page.

evaluation.py file

- ```
input_path = "/opt/ml/processing/input/test" #"output/test" #
classes_path = "/opt/ml/processing/input/classes"#"output/manifest/test.csv"
model_path = "/opt/ml/processing/model" #"model" #
output_path = '/opt/ml/processing/output' #"output" #
```

Initially, after importing all the necessary libraries, we added all the paths. Then, we standardised the images as shown below.

```
HEIGHT = 224
WIDTH = 224
DEPTH = 3
NUM_CLASSES = 8
```

- ```
def load_classes(file_name):
    .....
```

Here, we are loading the classes. We load the .json file that we have created through the ground truth process, where images are provided with the labels through human annotation.

- ```
def _parse_image_function(example):

```

Here, we are parsing the image and getting the image and its label in return.

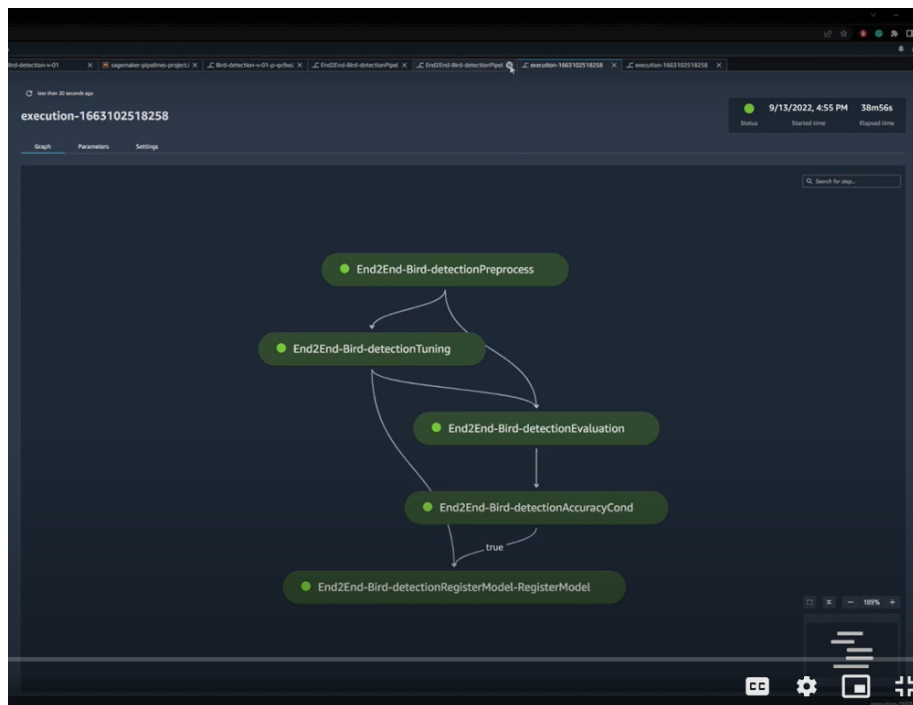
- ```
def predict_bird(model, img_array):
    .....
```

Through the use of this function, we are predicting the label using our model and making the prediction for an image.

- `hyperparameters = {'batch_size': 32,
 'data_dir': '/opt/ml/input/data'}`

The hyperparameters that we are setting are batch size and data drive or the path of the input data.

- Once the pipeline is completed and the machine has trained it, we will store the trained model in the S3 bucket output folder as shown below.
`bird_model_path =
's3://sagemaker-us-east-1-729987989507/a8hovs5dbbn8-End2End-3xrsS1pWuZ-001-7
d7393a3/output/model.tar.gz'`
- You can also see the run time of the pipeline if it ran successfully. You can also see the graph of the pipeline stages as shown below.



You can click on each stage and view the details of that particular stage, such as the input, output, logs and run time.

- `from sagemaker.tensorflow import TensorFlowModel
TF_FRAMEWORK_VERSION = '2.4.1'`

```
model = TensorFlowModel(
    model_data=bird_model_path,
    role=role,
    framework_version=TF_FRAMEWORK_VERSION)
```

```
predictor = model.deploy(serverless_inference_config=serverless_inf_config)
tf_endpoint_name = str(predictor.endpoint_name)
print(f"Endpoint [{predictor.endpoint_name}] deployed")
```

Here, as you can see, we have imported the TensorFlow Model. We set the path of the saved model as *bird_model_path*. Then, we deploy the endpoint. It takes a few minutes for this to run and deploy. You will get the name of the endpoint, which you can also see from the dropdown menu in the Endpoints option. Please ensure you enter the correct endpoint name in the variable in the next cell at line.

```
tf_endpoint_name = 'tensorflow-inference-2022-09-22-10-29-12-372'
```

- Utility function is used to create the test data point for our pipeline.

```
import cv_utils
sample_images =
cv_utils.get_n_random_images(default_bucket,prefix=f'{base_job_prefix}/outputs/test',n=
1)

local_paths = cv_utils.download_images_locally(default_bucket,sample_images)
```

Here, you provided 1 image to the machine for carrying out the prediction. We gave the path to the test data set and set *n=1* for fetching the image. You can use multiple images as well.

- *for inputfile in local_paths:*

```
cv_utils.predict_bird_from_file(inputfile,predictor,possible_classes)
```

This loop will predict the class of the image as shown below:

```
Class: 013.Bobolink, Confidence :1.00
./inference-test-data/Bobolink_0001_9261.jpg
```

- Please ensure that you delete the endpoints and remaining resources once you are done with your project so you are not charged unnecessarily for them.